

Dealing with Compound Words

Unfortunately, two valid words appearing together without a space is not conclusive evidence that a space is missing. Consider the word *butterfly*. We know that *butter* is a valid word and *fly* is a valid word, but we can't necessarily conclude that *butterfly* was written in error, because *butterfly* is also a valid word. So we need to check not only for valid words that appear together without a space but also for valid words that, when mashed together without a space, do not together form another valid word. This means that in our text, we need to check whether `oneperfectly` is a word, whether `paradisegiven` is a word, and so on.

In order to check this, we need to find all the spaces in our text. We can look at all the substrings between two consecutive spaces and call those potential words. If a potential word is not in our word list, then we'll conclude that it's invalid. We can check each invalid word to see whether it's made up of a combination of two smaller words; if it is, we'll conclude that there's a missing space and add it back in, right between the two valid words that have combined to form the invalid word.

Checking Between Existing Spaces for Potential Words

We can use `re.finditer()` again to find all the spaces in our text, which we'll store in a variable called `spacestarts`. We'll also add two more elements to our `spacestarts` variable: one to represent the location of the beginning of the text and one to represent the location of the end. This ensures that we find every potential word, since words at the very beginning and end will be the only words that are not between spaces. We also add a line that sorts the `spacestarts` list:

```
spacestarts = [m.start() for m in re.finditer(' ', text)]
spacestarts.append(-1)
spacestarts.append(len(text))
spacestarts.sort()
```

The list `spacestarts` records the locations of the spaces in our text. We got these locations by using a list comprehension and the `re.finditer()` tool. In this case, `re.finditer()` finds the location of every space in the text and stores it in a list, which refers to each individual element as `m`. For each of those `m`